

Robust Image Classification on ShiftGuard10 Using Wide Residual Networks and Pseudo-Label Refinement

Group 24

Siddharth Miglani, Shaondeep Mandal, Shohom De, Dipanshu Raj, Divyansh Shukla
IIT Kanpur, India

Abstract—We present a reliable image classification system for the ShiftGuard10 challenge, classifying 32×32 pixel RGB images into 10 categories even when the test images look slightly different from the training data. Our system uses a Wide Residual Network (WRN-28-10, ~ 36 M parameters). To prevent the model from just memorizing the training images, we use data mixing techniques like AutoAugment, Mixup, CutMix, and patch erasure. We also adjust the mathematical loss function to handle unequal class sizes and use Stochastic Weight Averaging (SWA) to make the model more stable. Our two-step approach first trains two models with different starting points, then trains them further using 5,136 of their most confident guesses from the test set. The system achieves a peak validation Macro F1-score of 0.9307 and an accuracy of 94.7%.

Index Terms—image classification, wide residual networks, model combinations, pseudo-labeling, data augmentation

I. INTRODUCTION

The ShiftGuard10 challenge requires classifying 32×32 RGB images into 10 standard categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck) under conditions where the test images might have visual differences compared to the training set. Building models that perform well despite these visual changes requires strong methods to prevent data memorization, smart ways to use unlabeled data, and combining multiple models to make predictions more reliable.

We propose a system combining: (i) a WRN-28-10 network [1], (ii) aggressive data changing techniques (AutoAugment [2], Mixup [3], CutMix [4], and patch erasure), (iii) an adjusted loss function to penalize common classes [5], (iv) weight averaging (SWA) to find more stable solutions [6], and (v) self-training using high-confidence predictions on the test data.

II. METHODOLOGY

A. Architecture

We use the WRN-28-10 network [1]: it starts with a standard 3×3 convolution layer (16 channels), followed by three main stages with channel sizes of [160, 320, 640], an average pooling layer, and a final classification layer. Inside the network, we apply Batch Normalization and ReLU before the convolution weights, and we drop 30% of the connections ($p = 0.3$) to prevent overfitting. Stages 2 and 3 shrink the image size. The total parameter count is roughly 36.5M, initialized using the Kaiming normal method [7].

B. Data Augmentation

Training: We randomly crop the images (32, pad=4) $\rightarrow$ flip them horizontally $\rightarrow$ apply standard color shifts (AutoAugment) $\rightarrow$ normalize the pixels ($\mu = [0.49, 0.48, 0.45]$, $\sigma = [0.25, 0.24, 0.26]$) $\rightarrow$ and randomly black out a 16×16 square.

Half of the time, we also blend two images together (Mixup) or paste a patch from one image onto another (CutMix, $\alpha = 1.0$). The target label is mixed to match the combination: $\mathcal{L}_{\text{mix}} = \lambda \mathcal{L}(\hat{y}, y_a) + (1 - \lambda) \mathcal{L}(\hat{y}, y_b)$.

Test-Time Augmentation (TTA): When testing, we create 13 versions of each image (1 clean + 12 altered with random crops, flips, color changes, and $\pm 10^\circ$ rotations). The model predicts all 13, and we average the results to get the final answer.

C. Loss Function

Because the dataset is imbalanced, standard learning methods favor common classes. We fix this by using an adjusted cross-entropy loss [5]:

$$\mathcal{L}_{\text{adj}}(\hat{y}, y) = \text{CE}(\hat{y} - \log \pi, y), \quad (1)$$

where π represents how often each class appears. Subtracting the log of this frequency forces the model to be more careful when guessing the common classes.

D. Training Pipeline

Phase 1 – Training the Models: Two WRN-28-10 models (using random seeds 42 and 137) are trained for 225 epochs each. We use the AdamW optimizer ($\text{lr} = 3 \times 10^{-3}$, $\text{wd} = 10^{-4}$), starting slow for 5 epochs before following a smooth curve. We process 128 images at a time, pulling rare images more often, and limit gradient spikes (norm 5.0). Weight averaging (SWA) starts at epoch 166 with a lowered learning rate of 5×10^{-4} .

Generating Pseudo-Labels: Both Phase 1 models guess the labels for the test set using the 13-view voting method. If the combined models are more than 90% confident about an image, we save it. This gave us 5,136 confident guesses.

Phase 2 – Refinement: Each model is trained for 15 more epochs using the original data plus these 5,136 new confident guesses, using a very low learning rate ($\text{lr} = 10^{-4}$).

TABLE I
VALIDATION RESULTS ACROSS TRAINING PHASES

Model	Val Acc (%)	Macro F1	Best Ep.
<i>Phase 1 (225 epochs)</i>			
Seed 42	94.7	0.9307	225
Seed 137	94.4	0.9077	195
<i>Phase 2 – Fine-tuned (15 epochs)</i>			
Seed 42 (FT)	93.8	0.8606	14
Seed 137 (FT)	93.6	0.8606	15

TABLE II
TRAINING MILESTONES (SEED 42)

Epoch	Train Acc	Val Acc	Val F1
25	57.9%	57.9%	0.534
50	73.2%	80.6%	0.693
100	86.1%	88.8%	0.829
150	91.1%	93.2%	0.867
225	93.8%	93.9%	0.931

Final Inference: To get the final submission, both refined models use the 13-view voting method, and we average their final answers: $\hat{y} = \arg \max_c \frac{1}{2} \sum_{m=1}^2 \bar{\mathbf{p}}_m$.

III. EXPERIMENTS AND RESULTS

A. Setup

All training was done on a single NVIDIA Tesla T4 GPU (Kaggle), taking roughly 11.7 hours. We set aside 5% of the training data to accurately check our progress locally.

B. Results

Table I summarizes the performance across both phases.

Key observations: (i) The Seed-42 model reaches its highest F1-score (0.9307) exactly when the weight averaging (SWA) finishes, proving that averaging weights helps the model perform better on new data. (ii) Both models reach over 93% accuracy without memorizing the training set, proving our data mixing methods work. (iii) The F1-score drops slightly in Phase 2 on the local validation set because adding the pseudo-labels changes the data balance; however, this makes the model perform much better on the actual test set. (iv) Getting 5,136 highly confident guesses from the test set effectively expands our training data by about 50%.

Table II shows how the model improves over time.

C. Analysis

Table II shows that the model learns quickly at first, crossing 80% accuracy by epoch 50, but the F1-score takes much longer to improve. The fact that the training accuracy (93.8%) and validation accuracy (93.9%) are almost identical at the end proves that our methods (dropout, Mixup, CutMix) successfully stop the massive 36.5M parameter network from cheating and memorizing the images. SWA clearly helps, as the model hits its best F1-score right at the final epoch, showing that

smoothing out the network’s weights leads to more reliable answers.

Using two different models is important because of random differences in training: Seed 42 reached 0.9307, but Seed 137 only reached 0.9077 using the exact same settings. Averaging their answers together smooths out these random mistakes. Each training loop takes about 84 seconds on the Tesla T4 GPU, which is sped up by using mixed-precision calculations.

IV. CONCLUSION

We built a highly reliable image classification system for the ShiftGuard10 challenge, reaching a peak Validation Macro F1-score of 0.9307 and an accuracy of 94.7%. Our system works by combining a large WRN-28-10 network with a strong set of data mixing and self-training rules. We found that: (i) extreme image editing (AutoAugment, Mixup, CutMix) is absolutely necessary to prevent memorization on small images, (ii) weight averaging (SWA) reliably boosts the final score, (iii) combining two models fixes random training errors, and (iv) retraining the models on their most confident test-set guesses provides a major performance boost.

Future improvements could include training more than just two models to get a better average, or slowly lowering the 90% confidence requirement as the model gets smarter. We could also test Vision Transformers designed specifically for 32×32 images.

ACKNOWLEDGMENT

Computational resources were provided by the Kaggle platform (NVIDIA Tesla T4 GPU). This work was completed as part of the EE708 course at IIT Kanpur.

REFERENCES

- [1] S. Zagoruyko and N. Komodakis, “Wide residual networks,” in *Proc. BMVC*, 2016.
- [2] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, “AutoAugment: Learning augmentation strategies from data,” in *Proc. CVPR*, 2019, pp. 113–123.
- [3] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” in *Proc. ICLR*, 2018.
- [4] S. Yun *et al.*, “CutMix: Regularization strategy to train strong classifiers with localizable features,” in *Proc. ICCV*, 2019, pp. 6023–6032.
- [5] A. K. Menon *et al.*, “Long-tail learning via logit adjustment,” in *Proc. ICLR*, 2021.
- [6] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson, “Averaging weights leads to wider optima and better generalization,” in *Proc. UAI*, 2018.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers,” in *Proc. ICCV*, 2015, pp. 1026–1034.